🎙️ Project Vartalaap v2.0: Indic-Aware Hybrid Turn Detection Engine

Acoustic-Prosodic & Semantic Turn Coordination with Pipecat Smart-Turn v3.2, Gnani Conversational Stack, and Speculative Soft-Ducking Barge-In

1. Executive Summary & Core Architectural Upgrade (v2.0)

Vartalaap v2.0 directly solves the four fundamental audio-duplex and latency bottlenecks inherent in traditional voice bot architectures:
1. **The Streaming STT Latency Trap on Barge-Ins:** Waiting for cloud ASR to transcribe incoming user speech before deciding whether to interrupt introduces a 300ms–600ms latency penalty.
2. **Text-Only Blindness to Pitch Contours:** Questions and assertions in Hindi, Tamil, and Telugu frequently share identical syntactic order (*"kar diya..."* vs *"kar diyaaa?"*). Pitch contour and vowel elongation carry the turn boundary, which pure text tokenizers miss.
3. **Acoustic Echo Loops During Playback:** Bot audio leaking into the microphone triggers false VAD speech detections.
4. **TTFB Dead-Air:** Waiting until the entire silence threshold expires before triggering the LLM adds unavoidable latency to the conversation.

The v2.0 Solution: Dual-Stream Neuro-Symbolic Gate

- **Layer 1: Acoustic VAD (CPU):** Silero VAD (<2ms) operating on 30ms frames from a rolling 8.0-second PCM ring buffer.
- **Layer 2A: Deep Prosody & Acoustic Turn Classifier (RTX 5060):** Pretrained **Pipecat Smart-Turn v3.2 (32MB ONNX/PyTorch)** executing in `<8ms` to compute $P_{\text{acoustic}}(\text{incomplete})$.
- **Layer 2B: Real-Time Streaming ASR:** Gnani Prisma STT yielding interim tokens to compute lexical postposition and conjunction penalties $S_{\text{lexical_penalty}}$.
- **Layer 3A: Two-Stage Cultural Backchannel Gate with Soft-Ducking:**
- *Stage 1 ($t = 2\text{ms}$ on voice detection):* Instantly **soft-ducks** bot output by **-18 dB** (prevents harsh cutoffs).
- *Stage 2 ($t = 180\text{ms}–320\text{ms}$):* If duration $<450\text{ms}$ and token $\in$ Indic Backchannel Lexicon (*"haanji"*, *"achha"*, *"seri"*), **restores volume to 0 dB** (`BACKCHANNEL_IGNORED`). If duration $>500\text{ms}$ or genuine command (*"ruko"*, *"cancel"*), triggers a **hard flush** in $<60\text{ms}$.
- **Layer 3B: Speculative LLM Prefetching:** If $P(\text{complete}) > 0.75$ at 200ms of silence, speculative token generation with Gnani Evon initiates in the background, slashing Time-to-First-Audio (TTFA).

2. Dynamic Silence Controller & Mathematical Formulation

Instead of discrete if-else step functions, the Dynamic Silence Controller calculates the **Expected Turn Silence (ETS)** continuously using fused acoustic-prosodic and lexical probabilities:

$$\text{ETS(ms)} = 250\text{ms} + (2000\text{ms} - 250\text{ms}) \times \text{clamp}\Big(P_{\text{acoustic}}(\text{incomplete}) + S_{\text{lexical_penalty}}, 0.0, 1.0\Big)$$

Context Category	Indic Linguistic & Acoustic Markers	Dynamic Window	Execution Action
Hanging Conjunction / Postposition	Trailing case markers (*ke*, *ka*, *ki*, *mein*, *se*, *ko*, *par*) or conjunctions (*aur*, *lekin*, *magar*, *kyunki*)	**1,800 ms**	Floor held. Incomplete clause detected; agent suppresses silence trigger.
Cognitive Hesitation & Filler	*"matlab..."*, *"woh kya hai na..."*, *"ante..."*, *"seri na..."*, vowel lengthening	**2,000 ms**	Floor held. Telemetry outputs `HOLDING: HESITATION`.
Syntactic Incompleteness	Incomplete SOV sentence structure; rising intonation contour	**1,400 ms**	Acoustic model flags incomplete boundary; extends timeout window.
Clean Closure (Terminal)	Terminal verb closure (*"karo"*, *"hai"*, *"chahiye"*) + falling pitch	**300 ms**	Fast turn handoff. Speculative LLM tokens sent to TTS immediately.
Short Confirmation	*"Haan"*, *"Nahi"*, *"Theek hai"*, *"Yes"* (when floor is open)	**250 ms**	Near-instantaneous turnaround for fast confirmation turns.
Mid-Turn Backchannel	User says *"Haanji"*, *"Achha"*, *"Seri"*, *"Hmm"* while bot plays (<85% progress)	**N/A**	Two-Stage Soft-Ducking restores playback to 0dB without terminating speech.
Genuine Barge-In	User says *"Ruko"*, *"Cancel karo"*, *"Stop"*, or long interruption (>500ms)	**< 60 ms**	Immediate hard audio cancellation and state rollback.

3. Direct Integration of `pipecat-ai/smart-turn-data-v3.2`

1. **Zero-Disk Streaming Evaluation Harness:**
2. Evaluated and benchmarked via Hugging Face `datasets` in `streaming=True` mode, filtering for Indic language tags (`hi`, `mr`, `bn`, `en`) and filler split partitions (`orpheus_midfiller_1`, `orpheus_endfiller_1`).
3. **Audio-Native Prosody Backbone:**
4. Feeds raw 16kHz PCM audio up to 8.0 seconds directly from a rolling memory ring buffer into the pretrained Smart-Turn v3.2 model on the RTX 5060 (<8ms), bypassing STT serialization lag.
5. **Domain-Adapted Calibration:**
6. Smart-Turn activation thresholds calibrated against 30 recorded Indic code-mixed hesitation samples to tune sensitivity for South Asian cadences.

4. Revised 8-Day Implementation Roadmap (Nov 2 – Nov 10)

Phase	Primary Objectives	Deliverables & Verification Gate
Phase 1: Foundations & APIs *(Days 1-2: Nov 2-3)*	• Set up Python 3.11 virtual env with `uv`. • Build `services/gnani_stt.py` & `services/gnani_tts.py`. • Download Smart-Turn v3.2 ONNX weights (32MB). • Initialize streaming harness on `smart-turn-data-v3.2-train`.	• Verified Gnani WebSocket auth (`X-API-Key-ID`). • Benchmark Smart-Turn inference latency (<8ms on RTX 5060). • Functional Timbre v3.0 voice streaming client.
Phase 2: Hybrid Engine & Backchannel *(Days 3-4: Nov 4-5)*	• Implement 8s rolling PCM ring buffer in `core/vad.py`. • Build Two-Stage Soft-Ducking Gate in `core/backchannel.py`. • Build Indic lexical postposition parser in `core/semantic.py`. • Implement continuous ETS state machine with speculative LLM prefetch.	• Cross-script normalizer (Romanized & native Indic backchannels). • Verified barge-in ducking (-18dB) with <60ms hard flush. • End-to-end turn decision loop executing in <15ms.
Phase 3: Pipeline & Frontend HUD *(Days 5-6: Nov 6-7)*	• Integrate FastAPI WebSocket orchestrator (`app/main.py`). • Implement WebRTC browser AEC and noise cancellation. • Build Real-Time Telemetry HUD (HTML5 + Tailwind + Canvas). • Add live A/B switch: Standard 500ms VAD vs Vartalaap.	• Dual audio visualizer displaying real-time ducking levels. • Live telemetry HUD: P_{acoustic} gauge, dynamic timer, TTFA. • Full-duplex audio loop verified without self-interruption.
Phase 4: Two-Tier Benchmark & Submission *(Days 7-8: Nov 8-10)*	• Execute Tier A benchmark ($N=500$, Smart-Turn test split). • Execute Tier B benchmark ($N=30$, Indic code-mix stress suite). • Record 3-minute split-screen demo video. • Finalize documentation and repository.	• Exported benchmark report verifying <5% cut-off rate. • High-quality demo showcasing *"Haanji"* soft-ducking recovery live. • Final submission package ready before Nov 10, 23:59 IST.

5. Repository Structure (v2.0)

```
c:\Antigravity Dev\Gnani Hackathon\
├── .env                              # Gnani API keys & model endpoints
├── requirements.txt                  # fastapi, uvicorn, torch, torchaudio, silero-vad, datasets, onnxru
├── README.md                         # Architecture, benchmarks, reproduction steps
├── benchmark/
│   ├── run_benchmark.py              # Two-tier evaluation suite (Tier A: HF test set, Tier B: Hinglish
│   ├── test_samples/                 # 30 curated Hinglish, Tanglish, and Telugu stress-test audio files
│   └── results_summary.json          # Quantitative outputs: cut-off rates, dead-air time, false stops
├── app/
│   ├── __init__.py
│   ├── main.py                       # FastAPI WebSocket server managing full-duplex audio queues
│   ├── config.py                     # System thresholds, ETS limits, and device routing configs
│   ├── core/
│   │   ├── __init__.py
│   │   ├── vad.py                    # Silero VAD frame classifier + 8.0s rolling PCM audio ring buffer
│   │   ├── engine.py                 # Hybrid turn-taking orchestrator & speculative prefetch coordinato
│   │   ├── backchannel.py            # Two-Stage Soft-Ducking Cultural Gate (Hindi/Tamil/Telugu lexicons
│   │   ├── semantic.py               # Indic postposition & conjunction parser + Smart-Turn v3.2 wrapper
│   │   └── normalizer.py             # Cross-script phonetic normalizer (Devanagari/Tamil/Telugu/Roman)
│   ├── services/
│   │   ├── __init__.py
│   │   ├── gnani_stt.py              # Gnani Prisma STT WebSocket streaming client
│   │   ├── gnani_tts.py              # Gnani Timbre v3.0 streaming client with inline prosody tags
│   │   └── llm_brain.py              # Gnani Evon / Conversational agent generator with async cancellati
│   └── static/
│       ├── index.html                # Diagnostic Telemetry HUD (Audio waveforms, state pills, A/B toggl
│       ├── app.js                    # Web Audio API, WebRTC echo-cancellation constraints, WebSockets
│       └── styles.css                # Professional dark/light interface telemetry styling
└── docs/                             # Pre-build specification suite
    ├── 1_PRD.md
    ├── 2_TECHNICAL_ARCHITECTURE.md
    ├── 3_SECURITY_AND_ACCESS.md
    ├── 4_FRONTEND_SPECIFICATION.md
    ├── 5_FEATURE_TICKETS.md
    └── AGENT_WORKFLOW.md
```